

# Mobile Application Development

Comprehensive Lecture Notes & Study Guide  
Weeks 1–8 | 16 Lectures

|               |                                            |
|---------------|--------------------------------------------|
| Institution   | Riphah International College (RIC), Lahore |
| Department    | Computer Science                           |
| Course        | Mobile Application Development             |
| Prepared by   | Lecture Team, CS Department                |
| Academic Year | 2024–2025                                  |
| Format        | Flutter · React Native · UI/UX · Firebase  |

## Table of Contents

---

### Week 1

- › Lect 1: Introduction to Mobile App Development
- › Lect 2: Setting Up Development Environment

### Week 2

- › Lect 3: Flutter – Widgets, Layouts & State
- › Lect 4: React Native – Components, Props & State

### Week 3

- › Lect 5: UI/UX Design Principles
- › Lect 6: Building Responsive UIs

### Week 4

- › Lect 7: Navigation in Flutter
- › Lect 8: Navigation in React Native

### Week 5

- › Lect 9: State Management in Flutter
- › Lect 10: State Management in React Native

### Week 6

- › Lect 11: RESTful APIs & JSON
- › Lect 12: Integrating APIs in Flutter & React Native

### Week 7

- › Lect 13: Local Database & Storage
- › Lect 14: Firebase Integration

### Week 8

- › Lect 15: Advanced UI – Flutter Animations
- › Lect 16: Advanced UI – React Native Animations

WEEK 1 – Introduction to Mobile Application Development

Lecture 1: Introduction to Mobile Application Development

1.1 Course Overview & Objectives

This course provides a thorough foundation in modern mobile application development using industry-standard frameworks. By the end, students will be able to design, build, and deploy mobile applications for both Android and iOS.

- Understand the mobile development ecosystem and platform differences.
- Build cross-platform apps using Flutter and React Native.
- Implement navigation, state management, API integration, and database connectivity.
- Follow UI/UX best practices for professional-grade interfaces.
- Integrate cloud services (Firebase) and local databases.
- Deploy and test apps on Android and iOS simulators.

1.2 Importance of Mobile App Development

Mobile phones have become the primary computing device for billions of people. In 2024, global smartphone users exceeded 6.8 billion. Mobile apps drive revenue across e-commerce, healthcare, education, and entertainment.

| Metric                         | Value                      |
|--------------------------------|----------------------------|
| Global Smartphone Users (2024) | ~6.8 Billion               |
| Google Play Store Apps         | 3.5 Million+               |
| Apple App Store Apps           | 1.8 Million+               |
| Mobile Internet Traffic        | ~60% of all web traffic    |
| Average Apps per Smartphone    | 80 installed, 9 used daily |

1.3 Overview of Mobile Platforms

**Android:** Open-source OS by Google, based on Linux kernel. Written primarily in Kotlin/Java. Dominant globally (~72% market share). Uses Android Studio IDE.

**iOS:** Proprietary OS by Apple. Written in Swift/Objective-C. Premium market segment (~27% share). Uses Xcode IDE (macOS only).

**Cross-Platform:** Write once, deploy to both platforms. Key frameworks:

- Flutter (Google) – Dart language, own rendering engine
- React Native (Meta) – JavaScript, bridges to native components
- Xamarin (Microsoft) – C#/.NET based

| Android           | iOS               | Cross-Platform           |
|-------------------|-------------------|--------------------------|
| Kotlin / Java     | Swift / Obj-C     | Dart / JavaScript        |
| Android Studio    | Xcode             | VS Code / Android Studio |
| Google Play Store | Apple App Store   | Both stores              |
| Any OS for dev    | macOS only        | Any OS                   |
| ~72% market share | ~27% market share | Both audiences           |

■ *Note: Focus of this course: Flutter and React Native as cross-platform solutions, giving you maximum reach with a single codebase.*

★ **ACTIVITY: Discussion: Name 5 popular apps and identify their platform (Android/iOS/Both). Which framework do you think they used?**

## Lecture 2: Setting Up the Development Environment

### 2.1 Installing IDEs

**Android Studio** – Official IDE for Android development. Required for Flutter Android builds.

- Download from [developer.android.com/studio](https://developer.android.com/studio)
- Install Android SDK, Android Virtual Device (AVD) Manager
- Configure ANDROID\_HOME environment variable
- Create a new AVD for testing (e.g., Pixel 6, API 34)

**Xcode** – Required for iOS development (macOS only).

- Install from Mac App Store (free)
- Accept license: `sudo xcodebuild -license`
- Install iOS simulators via Xcode > Preferences > Components

**VS Code** – Lightweight, cross-platform editor. Recommended for React Native.

- Flutter extension (Dart support)
- React Native Tools
- ESLint, Prettier

### 2.2 Introduction to Flutter

Flutter is Google's UI toolkit for building natively compiled apps from a single codebase. It uses the **Dart** programming language and has its own rendering engine (Skia/Impeller), meaning it doesn't rely on native UI components.

**Installation steps:**

- Download Flutter SDK from [flutter.dev](https://flutter.dev)
- Extract and add flutter/bin to PATH
- Run: `flutter doctor` (checks environment)
- Create project: `flutter create my_app`
- Run: `flutter run`

```
# Sample Flutter doctor output check
$ flutter doctor
[OK] Flutter (Channel stable)
[OK] Android toolchain
[OK] Android Studio
[!] Xcode - Install Xcode for iOS builds
```

### 2.3 Introduction to React Native

React Native allows building mobile apps using JavaScript and React. Unlike Flutter, it uses a **JavaScript bridge** to communicate with native components, meaning the UI looks and feels native on each platform.

- Install Node.js (LTS) from [nodejs.org](https://nodejs.org)
- Install React Native CLI: `npm install -g react-native-cli`
- Create project: `npx react-native init MyApp`
- Run Android: `npx react-native run-android`
- Run iOS: `npx react-native run-ios` (macOS only)

★ **ACTIVITY: Lab Task: Set up Flutter and create a 'Hello, World!' app. Run it on the Android emulator. Take a screenshot for your portfolio.**

## WEEK 2 – Fundamentals of Mobile App Development

### Lecture 3: Flutter – Widgets, Layouts & State Management

#### 3.1 Everything is a Widget

In Flutter, the entire UI is built from **widgets**. A widget is an immutable description of part of the user interface. Flutter has two main widget types:

- **StatelessWidget** – Does not change after creation. Used for static content.
- **StatefulWidget** – Can rebuild when internal state changes.

```
class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: Text('My App')),
        body: Center(child: Text('Hello, Flutter!')),
      ),
    );
  }
}
```

#### 3.2 Common Layout Widgets

| Widget    | Purpose              | Key Properties                                  |
|-----------|----------------------|-------------------------------------------------|
| Container | Box model element    | padding, margin, color, width, height           |
| Row       | Horizontal layout    | children, mainAxisAlignment, crossAxisAlignment |
| Column    | Vertical layout      | children, mainAxisAlignment                     |
| Stack     | Overlapping children | children, alignment                             |
| Expanded  | Fill available space | flex, child                                     |
| ListView  | Scrollable list      | children, builder, scrollDirection              |
| GridView  | Grid layout          | gridDelegate, children                          |

#### 3.3 State Management Basics

State is any data that can change and cause the UI to rebuild. In Flutter, calling **setState()** inside a StatefulWidget triggers a rebuild.

```
class CounterApp extends StatefulWidget {
  @override
  _CounterAppState createState() => _CounterAppState();
}

class _CounterAppState extends State {
  int _count = 0;
  void _increment() => setState(() => _count++);
  @override
  Widget build(BuildContext context) {
```

```
return Column(children: [
  Text('Count: $_count'),
  ElevatedButton(onPressed: _increment, child: Text('Increment')),
]);
}
```

★ **ACTIVITY: Assignment No. 1: Build a Flutter app with a counter that increments, decrements, and resets. Add a color that changes based on the count value.**

## Lecture 4: React Native – Components, Props & State

### 4.1 Core Concepts

React Native apps are built from **components** – reusable UI building blocks written as JavaScript functions or classes. Data flows down through **props** (properties), and dynamic data is managed via **state**.

### 4.2 Core Components

| Component        | Equivalent to            | Usage                |
|------------------|--------------------------|----------------------|
| View             | div (web) / UIView (iOS) | Container/layout     |
| Text             | p, span                  | Display text         |
| Image            | img                      | Display images       |
| TextInput        | input[type=text]         | Text entry           |
| ScrollView       | overflow: scroll         | Scrollable container |
| FlatList         | Virtualised list         | Efficient long lists |
| TouchableOpacity | button / clickable       | Pressable elements   |

### 4.3 Props and State Example

```
import React, { useState } from 'react';
import { View, Text, Button, StyleSheet } from 'react-native';

function Greeting({ name }) { // 'name' is a prop
  return Hello, {name}!;
}

export default function App() {
  const [count, setCount] = useState(0); // state
  return (

    Count: {count}
    setCount(count + 1)} />

  );
}
```

■ *Note: Key difference: Flutter uses Dart classes and setState; React Native uses JavaScript hooks (useState, useEffect).*



## WEEK 3 – User Interface (UI) Design

### Lecture 5: Principles of UI/UX Design

#### 5.1 UI vs UX

**UI (User Interface)** refers to the visual elements – buttons, colors, typography, icons, layouts.

**UX (User Experience)** refers to the overall feel – usability, flow, accessibility, satisfaction.

#### 5.2 Core Design Principles

- **Consistency:** Use the same patterns, colors, and terminology throughout the app.
- **Feedback:** Always respond to user actions (loading indicators, success/error messages).
- **Affordance:** Make interactive elements look clickable (shadows, borders, colors).
- **Hierarchy:** Use size, weight, and color to guide the user's eye to important content.
- **Accessibility:** Support screen readers, sufficient contrast, scalable text.
- **Simplicity:** Remove unnecessary elements. Every element must earn its place.

#### 5.3 Material Design (Android) vs Human Interface Guidelines (iOS)

| Aspect          | Material Design (Google)    | HIG (Apple)              |
|-----------------|-----------------------------|--------------------------|
| Design Language | Bold, colorful, layered     | Clean, minimal, flat     |
| Navigation      | Bottom nav bar / Drawer     | Tab bar at bottom        |
| Typography      | Roboto font family          | SF Pro font family       |
| Icons           | Filled / outlined icons     | SF Symbols               |
| Transitions     | Shared element transitions  | Slide / push transitions |
| Color           | Primary, Secondary, Surface | System colors (adaptive) |

★ **ACTIVITY: Quiz/Test 1: Identify Material Design vs HIG elements in 3 screenshots provided. Explain design decisions for each.**

## Lecture 6: Building Responsive UIs

### 6.1 Responsive Design in Mobile

Mobile devices come in many screen sizes. A responsive UI adapts to different screen dimensions, orientations, and pixel densities.

- Use relative units (flex, percentages) instead of fixed pixels
- MediaQuery (Flutter) / Dimensions (React Native) for screen size
- OrientationBuilder for landscape/portrait modes
- Flexible and Expanded widgets fill available space proportionally

### 6.2 Flutter Responsive Widgets

```
// MediaQuery example in Flutter
final width = MediaQuery.of(context).size.width;
final isTablet = width > 600;

Widget build(BuildContext context) {
  return isTablet
    ? Row(children: [SideBar(), MainContent()])
    : Column(children: [MainContent()]);
}
```

### 6.3 React Native Responsive Design

```
import { Dimensions, StyleSheet } from 'react-native';
const { width, height } = Dimensions.get('window');

const styles = StyleSheet.create({
  container: {
    padding: width * 0.05, // 5% of screen width
    fontSize: width > 400 ? 18 : 14,
  }
});
```

## WEEK 4 – Navigation and Routing

### Lecture 7: Navigation in Flutter

#### 7.1 Flutter Navigation Concepts

Flutter uses a **Navigator** widget that manages a stack of Route objects. Think of it like a stack of screens – you push a new screen on top and pop it off to go back.

#### 7.2 Basic Navigation

```
// Push a new route
Navigator.push(
  context,
  MaterialPageRoute(builder: (context) => SecondScreen()),
);

// Go back
Navigator.pop(context);

// Pass data to a screen
Navigator.push(context,
  MaterialPageRoute(builder: (_) => DetailScreen(item: myItem)));

// Return data from a screen
final result = await Navigator.push(context,
  MaterialPageRoute(builder: (_) => InputScreen()));
// result contains what was passed to Navigator.pop(context, data)
```

#### 7.3 Named Routes

Named routes allow defining app routes in one place, making large apps easier to manage.

```
MaterialApp(
  initialRoute: '/',
  routes: {
    '/': (ctx) => HomeScreen(),
    '/detail': (ctx) => DetailScreen(),
    '/settings': (ctx) => SettingsScreen(),
  },
);

// Navigate using name
Navigator.pushNamed(context, '/detail');

// With arguments
Navigator.pushNamed(context, '/detail', arguments: {'id': 42});
```

## Lecture 8: Navigation in React Native

### 8.1 React Navigation Library

React Native does not have built-in navigation. The most popular library is **React Navigation**. Install it with:

```
npm install @react-navigation/native
npm install @react-navigation/stack
npm install react-native-screens react-native-safe-area-context
```

### 8.2 Stack Navigator

```
import { NavigationContainer } from '@react-navigation/native';
import { createStackNavigator } from '@react-navigation/stack';

const Stack = createStackNavigator();

export default function App() {
  return (

    );
}

// Navigate from HomeScreen:
function HomeScreen({ navigation }) {
  return (
    <TouchableOpacity onPress={() => navigation.navigate('Detail', { id: 1 })} />
  );
}
```

### 8.3 Tab Navigation

```
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
const Tab = createBottomTabNavigator();

function MyTabs() {
  return (

    );
}
```

## WEEK 5 – State Management

### Lecture 9: State Management in Flutter

#### 9.1 Why State Management Matters

As apps grow, managing state with `setState()` becomes complex. We need ways to share state across widgets and keep the code maintainable.

#### 9.2 Provider Package

Provider is the officially recommended lightweight state management solution for Flutter.

```
// 1. Create a ChangeNotifier
class CounterModel extends ChangeNotifier {
  int _count = 0;
  int get count => _count;
  void increment() { _count++; notifyListeners(); }
}

// 2. Wrap app with ChangeNotifierProvider
ChangeNotifierProvider(
  create: (_) => CounterModel(),
  child: MyApp(),
)

// 3. Consume in a widget
Consumer(
  builder: (ctx, model, child) => Text('${model.count}'),
)
```

#### 9.3 Riverpod

Riverpod is an improved version of Provider, offering compile-time safety, better testability, and no need for `BuildContext`.

```
// Define a provider
final counterProvider = StateNotifierProvider(
  (ref) => CounterNotifier(),
);

class CounterNotifier extends StateNotifier {
  CounterNotifier() : super(0);
  void increment() => state++;
}

// Use in a widget
class MyWidget extends ConsumerWidget {
  Widget build(BuildContext context, WidgetRef ref) {
    final count = ref.watch(counterProvider);
    return Text('$count');
  }
}
```

**★ ACTIVITY: Assignment No. 2: Refactor a simple Flutter shopping cart app to use Provider. Cart total must update across multiple screens.**

## Lecture 10: State Management in React Native

### 10.1 useState and useContext

```
// useState - local state
const [user, setUser] = useState(null);

// useContext - shared state
const AuthContext = createContext();

function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const login = (userData) => setUser(userData);
  const logout = () => setUser(null);
  return (
    {children}
  );
}

// In any child component:
const { user, login } = useContext(AuthContext);
```

### 10.2 Redux (Advanced)

Redux is a predictable state container for JavaScript apps. It centralises state and makes changes predictable through pure functions called **reducers**.

| Concept  | Description                                              |
|----------|----------------------------------------------------------|
| Store    | Single source of truth – holds entire app state          |
| Action   | Plain JS object describing what happened {type, payload} |
| Reducer  | Pure function: (state, action) => newState               |
| Dispatch | Sends an action to the store                             |
| Selector | Function to extract data from the store                  |

## WEEK 6 – Working with APIs

### Lecture 11: RESTful APIs & JSON

#### 11.1 What is a REST API?

REST (Representational State Transfer) is an architectural style for designing networked applications. A REST API exposes resources via URLs and uses HTTP methods to perform operations.

| HTTP Method | Operation              | Example URL         |
|-------------|------------------------|---------------------|
| GET         | Read/retrieve data     | GET /api/users/1    |
| POST        | Create new resource    | POST /api/users     |
| PUT         | Update entire resource | PUT /api/users/1    |
| PATCH       | Partial update         | PATCH /api/users/1  |
| DELETE      | Remove resource        | DELETE /api/users/1 |

#### 11.2 JSON Format

JSON (JavaScript Object Notation) is the standard data format for APIs. It is human-readable and language-independent.

```
{
  "id": 1,
  "name": "Ali Raza",
  "email": "ali@example.com",
  "courses": ["Flutter", "React Native"],
  "isEnrolled": true
}
```

★ **ACTIVITY: Quiz/Test 2: Identify HTTP methods and status codes (200, 201, 400, 401, 404, 500) from given API scenarios.**

## Lecture 12: Integrating APIs in Flutter & React Native

### 12.1 API Integration in Flutter (http package)

```
import 'package:http/http.dart' as http;
import 'dart:convert';

Future<List<User>> fetchUsers() async {
  final response = await http.get(
    Uri.parse('https://jsonplaceholder.typicode.com/users'),
    headers: {'Content-Type': 'application/json'},
  );
  if (response.statusCode == 200) {
    final List data = jsonDecode(response.body);
    return data.map((json) => User.fromJson(json)).toList();
  } else {
    throw Exception('Failed to load users');
  }
}
```

### 12.2 API Integration in React Native (axios)

```
import axios from 'axios';
import { useEffect, useState } from 'react';

function UserList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    axios.get('https://jsonplaceholder.typicode.com/users')
      .then(res => setUsers(res.data))
      .catch(err => console.error(err))
      .finally(() => setLoading(false));
  }, []);

  if (loading) return ;
  return (
    <div>
      {users.map(item => {item.name})}
    </div>
  );
}
```



## WEEK 7 – Database Integration

### Lecture 13: Local Storage Options

#### 13.1 Types of Local Storage

| Storage Type    | Flutter Package         | RN Package                  | Use Case                     |
|-----------------|-------------------------|-----------------------------|------------------------------|
| Key-Value Store | SharedPreferences       | AsyncStorage                | User settings, tokens        |
| Relational DB   | sqflite                 | react-native-sqlite-storage | Structured data              |
| Object Store    | Hive / Isar             | MMKV                        | Complex objects, fast access |
| File Storage    | path_provider + dart:io | react-native-fs             | Files, images                |

#### 13.2 SharedPreferences (Flutter)

```
import 'package:shared_preferences/shared_preferences.dart';

// Save data
final prefs = await SharedPreferences.getInstance();
await prefs.setString('username', 'ali_raza');
await prefs.setBool('isLoggedIn', true);

// Read data
final username = prefs.getString('username') ?? 'Guest';
final isLoggedIn = prefs.getBool('isLoggedIn') ?? false;

// Delete
await prefs.remove('username');
await prefs.clear(); // Clear everything
```

#### 13.3 SQLite in Flutter (sqflite)

```
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';

Future openDB() async {
  return openDatabase(
    join(await getDatabasesPath(), 'myapp.db'),
    onCreate: (db, version) {
      return db.execute(
        'CREATE TABLE students(id INTEGER PRIMARY KEY, name TEXT, grade TEXT)',
      );
    },
    version: 1,
  );
}

// Insert
await db.insert('students', {'name': 'Sara', 'grade': 'A'});

// Query
final students = await db.query('students');
```

```
// Update  
await db.update('students', {'grade': 'A+'}, where: 'id = ?', whereArgs: [1]);
```

## Lecture 14: Firebase Integration

### 14.1 What is Firebase?

Firebase is Google's Backend-as-a-Service (BaaS) platform. It provides a suite of tools for mobile and web apps including authentication, databases, storage, hosting, and analytics — without managing servers.

| Service           | Description                            | Common Use                   |
|-------------------|----------------------------------------|------------------------------|
| Authentication    | User sign-in with email, Google, phone | Login/Register screens       |
| Firestore         | NoSQL cloud database, real-time sync   | Chat, social feeds, products |
| Realtime Database | JSON tree, real-time sync              | Live scores, chat            |
| Cloud Storage     | File storage (images, videos)          | User avatars, uploads        |
| Cloud Messaging   | Push notifications                     | Alerts, reminders            |
| Analytics         | User behaviour tracking                | App insights                 |

### 14.2 Firebase Firestore – CRUD Operations (Flutter)

```
import 'package:cloud_firestore/cloud_firestore.dart';
final db = FirebaseFirestore.instance;

// Create a document
await db.collection('students').add({
  'name': 'Ahmed', 'roll': '101', 'gpa': 3.8
});

// Read a collection
final snapshot = await db.collection('students').get();
for (var doc in snapshot.docs) {
  print(doc['name']);
}

// Real-time listener
db.collection('students').snapshots().listen((snap) {
  for (var change in snap.docChanges) {
    print('${change.type}: ${change.doc.data()}');
  }
});

// Update
await db.collection('students').doc(docId).update({'gpa': 4.0});

// Delete
await db.collection('students').doc(docId).delete();
```

### 14.3 Firebase Authentication

```
import 'package:firebase_auth/firebase_auth.dart';
final auth = FirebaseAuth.instance;

// Register
await auth.createUserWithEmailAndPassword(
  email: 'student@ric.edu.pk', password: 'SecurePass123');
```

```
// Login
await auth.signInWithEmailAndPassword(
  email: 'student@ric.edu.pk', password: 'SecurePass123');

// Check auth state
final user = auth.currentUser;
if (user != null) print('Logged in as: ${user.email}');

// Logout
await auth.signOut();
```

## WEEK 8 – Advanced UI Components

### Lecture 15: Custom Widgets & Animations in Flutter

#### 15.1 Custom Widgets

Creating reusable custom widgets keeps your code DRY (Don't Repeat Yourself) and maintainable. Any piece of UI used more than once should become a widget.

```
class CustomCard extends StatelessWidget {  
  final String title;  
  final String subtitle;  
  final Color color;  
  final VoidCallback onTap;  
  
  const CustomCard({  
    required this.title, required this.subtitle,  
    this.color = Colors.blue, required this.onTap,  
  });  
  
  @override  
  Widget build(BuildContext context) {  
    return GestureDetector(  
      onTap: onTap,  
      child: Container(  
        padding: EdgeInsets.all(16),  
        decoration: BoxDecoration(  
          color: color.withOpacity(0.1),  
          borderRadius: BorderRadius.circular(12),  
          border: Border.all(color: color, width: 1.5),  
        ),  
        child: Column(  
          crossAxisAlignment: CrossAxisAlignment.start,  
          children: [  
            Text(title, style: TextStyle(fontWeight: FontWeight.bold)),  
            SizedBox(height: 4),  
            Text(subtitle, style: TextStyle(color: Colors.grey[600])),  
          ],  
        ),  
      ),  
    );  
  }  
}
```

#### 15.2 Flutter Animations

Flutter provides a rich animation system. Animation types include:

| Type               | Class/Widget                                        | Best For                              |
|--------------------|-----------------------------------------------------|---------------------------------------|
| Implicit Animation | AnimatedContainer, AnimatedOpacity, AnimatedPadding | Simple property changes               |
| Explicit Animation | AnimationController + Tween                         | Custom, looping, complex              |
| Hero Animation     | Hero widget                                         | Screen transition with shared element |

|        |                |                                              |
|--------|----------------|----------------------------------------------|
| Lottie | lottie package | Complex vector animations from After Effects |
|--------|----------------|----------------------------------------------|

```
// Implicit animation - fades in/out on tap
AnimatedOpacity(
  opacity: _visible ? 1.0 : 0.0,
  duration: Duration(milliseconds: 500),
  child: Text('Hello!'),
)

// Explicit animation with controller
class _MyState extends State with SingleTickerProviderStateMixin {
  late AnimationController _ctrl;
  late Animation _scale;

  @override void initState() {
    super.initState();
    _ctrl = AnimationController(vsync: this, duration: Duration(seconds: 1));
    _scale = Tween(begin: 0.5, end: 1.5).animate(
      CurvedAnimation(parent: _ctrl, curve: Curves.elasticOut));
    _ctrl.forward();
  }

  @override Widget build(BuildContext context) =>
    ScaleTransition(scale: _scale, child: Icon(Icons.star, size: 50));
}
```

## Lecture 16: Custom Components & Animations in React Native

### 16.1 Custom Components

```
import React from 'react';
import { View, Text, TouchableOpacity, StyleSheet } from 'react-native';

const CustomCard = ({ title, subtitle, color = '#1565C0', onPress }) => {
  return (
    { borderColor: color, backgroundColor: color + '18' ]}]>
    {title}
    {subtitle}

  );
};

const styles = StyleSheet.create({
  card: { padding: 16, borderRadius: 12, borderWidth: 1.5, marginVertical: 8 },
  title: { fontWeight: 'bold', fontSize: 16 },
  subtitle: { color: '#757575', marginTop: 4 },
});

export default CustomCard;
```

### 16.2 React Native Animations

React Native provides the **Animated API** for performant animations that run on the native thread.

```
import { Animated, Easing } from 'react-native';
import { useRef, useEffect } from 'react';

function FadeInView({ children }) {
  const fadeAnim = useRef(new Animated.Value(0)).current;

  useEffect(() => {
    Animated.timing(fadeAnim, {
      toValue: 1,
      duration: 1000,
      easing: Easing.ease,
      useNativeDriver: true, // Always set true for performance!
    }).start();
  }, [fadeAnim]);

  return (
    {children}

  );
}

// Spring animation example:
Animated.spring(scaleAnim, {
  toValue: 1.2,
  friction: 3,
  tension: 80,
  useNativeDriver: true,
}).start();
```

### 16.3 Reanimated 2 (Advanced)

For 60fps+ animations, use the **react-native-reanimated** library. It runs animations entirely on the UI thread.

```
import Animated, {
  useSharedValue, useAnimatedStyle, withSpring
} from 'react-native-reanimated';

function BounceButton() {
  const scale = useSharedValue(1);
  const animatedStyle = useAnimatedStyle(() => ({
    transform: [{ scale: scale.value }]
  }));
  return (
    <View>
      <Text>Press Me!</Text>
      <Animated.View>
        <Animated.View style={animatedStyle}>
          <Text>Press Me!</Text>
        </Animated.View>
      </Animated.View>
    </View>
  );
}
```

### Course Summary – Weeks 1–8

| Week | Topics Covered                                                           | Activity     |
|------|--------------------------------------------------------------------------|--------------|
| 1    | Intro to Mobile Dev, Platform Overview, IDE Setup, Flutter & RN Overview | —            |
| 2    | Flutter Widgets/Layouts, React Native Components/Props/State             | Assignment 1 |
| 3    | UI/UX Principles, Material Design vs HIG, Responsive UIs                 | Quiz/Test 1  |
| 4    | Flutter Navigator & Named Routes, React Navigation Library               | —            |
| 5    | Flutter Provider & Riverpod, React Native useState/useContext/Redux      | Assignment 2 |
| 6    | REST APIs, JSON, HTTP Methods, http (Flutter) & axios (RN)               | Quiz/Test 2  |
| 7    | Local Storage (SQLite, SharedPreferences, AsyncStorage), Firebase        | —            |
| 8    | Custom Widgets, Flutter Animations, React Native Animated API            | —            |

■ *Note: These notes cover all 16 lectures for Weeks 1–8. Students are encouraged to supplement with official Flutter ([docs.flutter.dev](https://docs.flutter.dev)) and React Native ([reactnative.dev](https://reactnative.dev)) documentation.*